

HULK SMASH Final Report

Department of Mechanical Engineering, Colorado State University

Group 2: Isabella Kaze, Brenton Yasuoka, Nathan Madachy

Lecture Section: 002

9 May 2025

TABLE OF CONTENTS

<i>DESIGN SUMMARY</i>	3
<i>SYSTEM DETAILS</i>	4
HARDWARE	8
<i>DESIGN EVALUATION</i>	11
<i>Output Display</i>	11
<i>Audio Output</i>	11
<i>Manual User Input</i>	12
<i>Automatic Sensors</i>	12
<i>Actuators, Mechanism, and Hardware</i>	13
<i>Logic, Processing, Control, Miscellaneous</i>	14
<i>Grading Adjustments</i>	14
<i>PARTIAL PART LIST</i>	18
<i>Gear Train System: 6:1 Gear Ratio</i>	18
<i>Chain: Number 35 Carbon Steel Roller Chain</i>	20
<i>Motor: Brushed 24V 150W 100RPM DC Motor</i>	21
HARDWARE	22
<i>LESSONS LEARNED</i>	27
<i>APPENDIX</i>	29

DESIGN SUMMARY

The Hulk Smash Can Crusher is a compact, automated device designed to safely and efficiently compress aluminum beverage cans for recycling (Figure 1). It simplifies the can disposal process by combining mechanical motion with sensor-based automation. The user inserts a can, and the device automatically detects and crushes it, providing audio and visual feedback throughout the process. To initiate crushing, the user selects the type of can from being crushed based on the drink type it was, and the system is able to track the crushing data over time. To physical crushing initiation, the machine signals a count down and plays “HULK, SMASH!” Once the can has been crushed sufficiently, it will slide through the tolerance slot and into the disposal receptacle. The system is intuitive, requiring minimal user effort while ensuring safe and reliable operation. To ensure safe operation, there is an emergency stop switch that will terminate all machine movement immediately and a reset button that returns the crushing head to the starting crushing position.

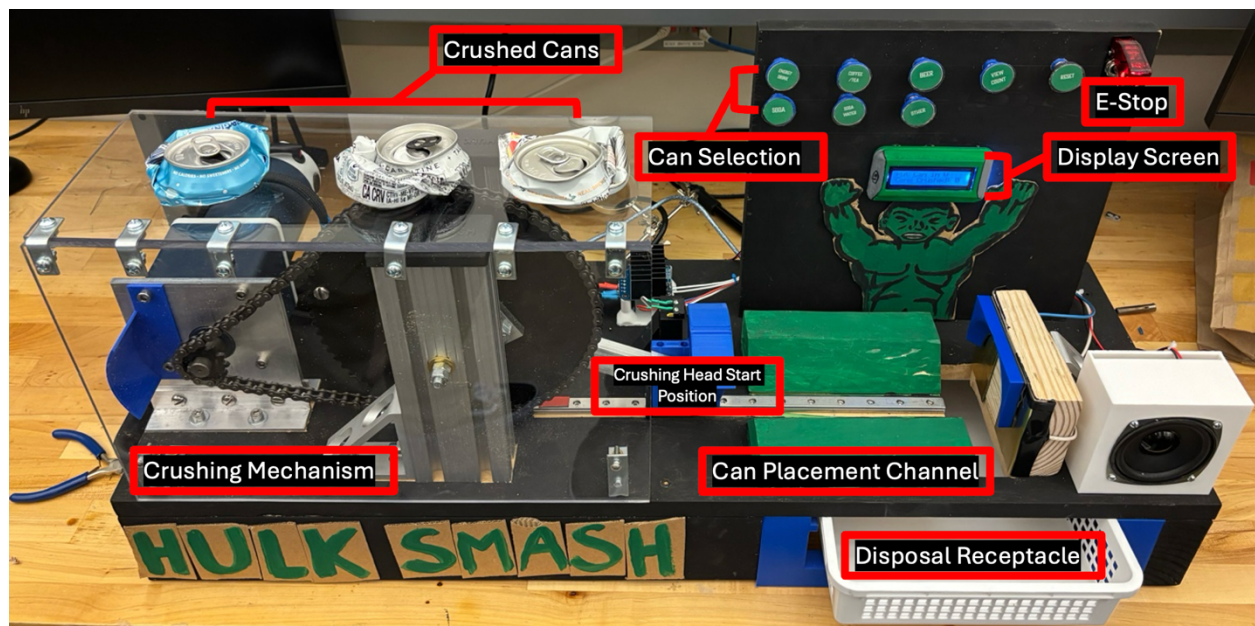


Figure 1: Front View of the Hulk Smash Can Crusher

SYSTEM DETAILS

The Hulk Smash can crushing machine is driven by a 24V gearhead motor that is attached to a crank-slider gear train system consisting of an input and output gear shown in Figure SD3. The crank-slider mechanism has an 8-inch stroke length allowing the operator to tall boy and standard sized aluminum cans. This geartrain system has a 6:1 gear ratio which provides adequate torque and speed to crush a can in under 5 seconds. The motor is powered by a 120V power supply which delivers a maximum of 240W to the motor unit.

To initiate the crushing cycle for the Hulk Smash machine, an automatic sensor is used to detect if an aluminum can is placed in the machine. The red cylinder seen in Figure SD2 is a 24 fluid oz. aluminum can which is the largest aluminum can the Hulk Smash machine accommodates. When the can is inserted into the machine, a two-plate conductivity sensor detects its presence by utilizing the aluminum can's conductive properties, closing the normally open circuit. After a can is detected, a signal is sent to the Arduino controller indicating a can has been placed inside the machine to be crushed. As seen in Figure SD1, bubble #4 displays two conductivity wires connected near the bottom of the frame where the can is inserted.

To properly read a signal from the conductivity sensor, an Arduino board is used to read the sensors and control the motor for the crushing cycle. This logic controller operates using an open-loop system and has a pre-determined function to crush a can once it is detected. In addition to receiving signals from the conductivity sensor, the Arduino controller will automatically count the number of crushed cans and present them on a liquid crystal display (LCD) screen. The LCD screen displays the total number of cans crushed by the machine which is updated by each successive crushing cycle. A limit switch located at the bottom-dead-center of the crank-slider stroke will count the cycles completed. One completed cycle will equate to one

additional can crushed which will be updated on the LCD screen. The operators will have a clear display of how many cans the machine has crushed in its operational lifetime. In addition, a button can be pressed at the beginning of the crushing cycle to explore the total number of each can type crushed.

While the operator is waiting for the machine to complete its cycle, a speaker mounted to the frame of the Hulk Smash machine will play a pre-recorded audio message. The audio from this speaker says, “Hulk Smash!!” in a Hulk-like voice. For each can type selected on the control board, this audio will play right before the crushing cycle begins. This audio recording is used as a warning to indicate the crushing cycle is about to begin. This serves to ensure the user keeps their hands clear from the mechanism along with entertaining them as they see the can get crushed.

Once the can has been crushed, the machine will return to its home position and pause to wait until another can is inserted into the V-channel. At any point during its operation, the user can engage an emergency stop to stop the motion of the crushing head. An arming switch is mounted to the front of the machine to rapidly stop the machine's motion. This red arming switch is intended to keep the machine in a safe state during the machine's operation. This normally open switch sends a signal to the Arduino board enabling or disabling power delivery to the motor. In the unlikely event of an emergency, this switch will kill all power to the motor preventing any further movement of the crushing head.

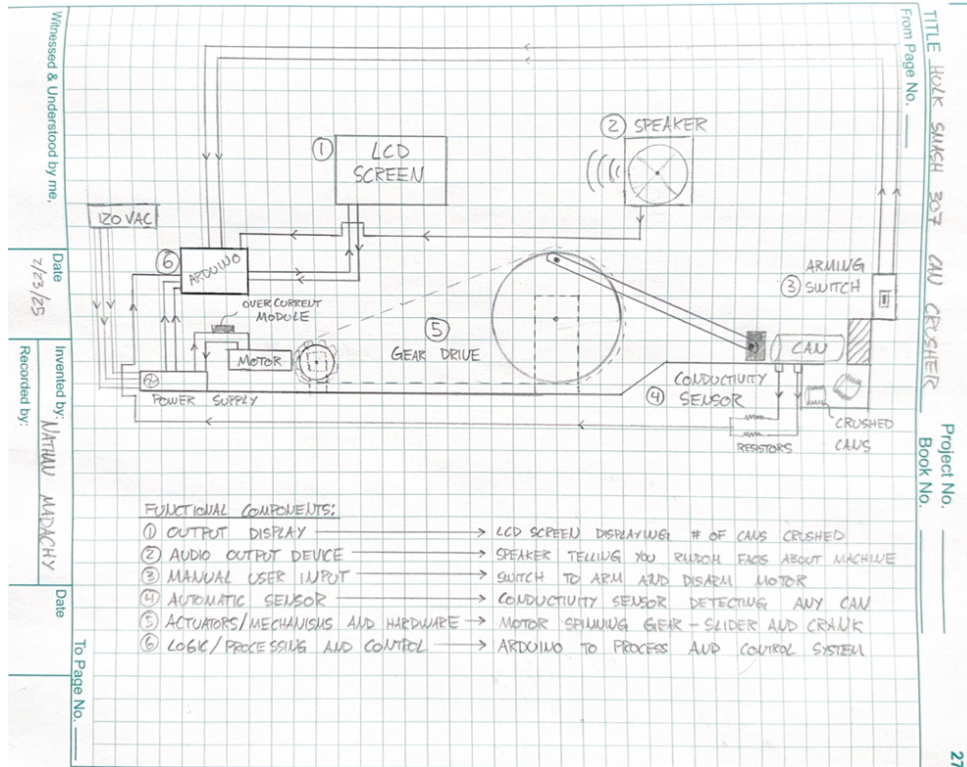


Figure SD1: Functional Diagram

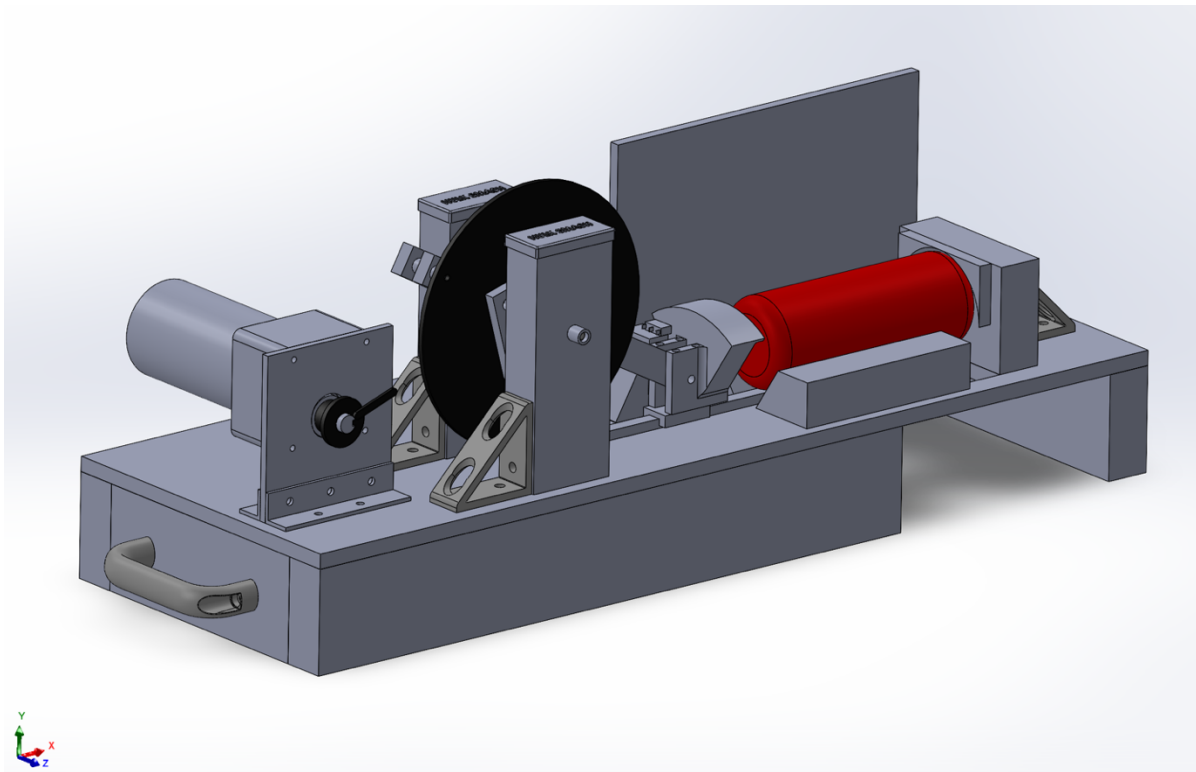


Figure SD2: SolidWorks Assembly

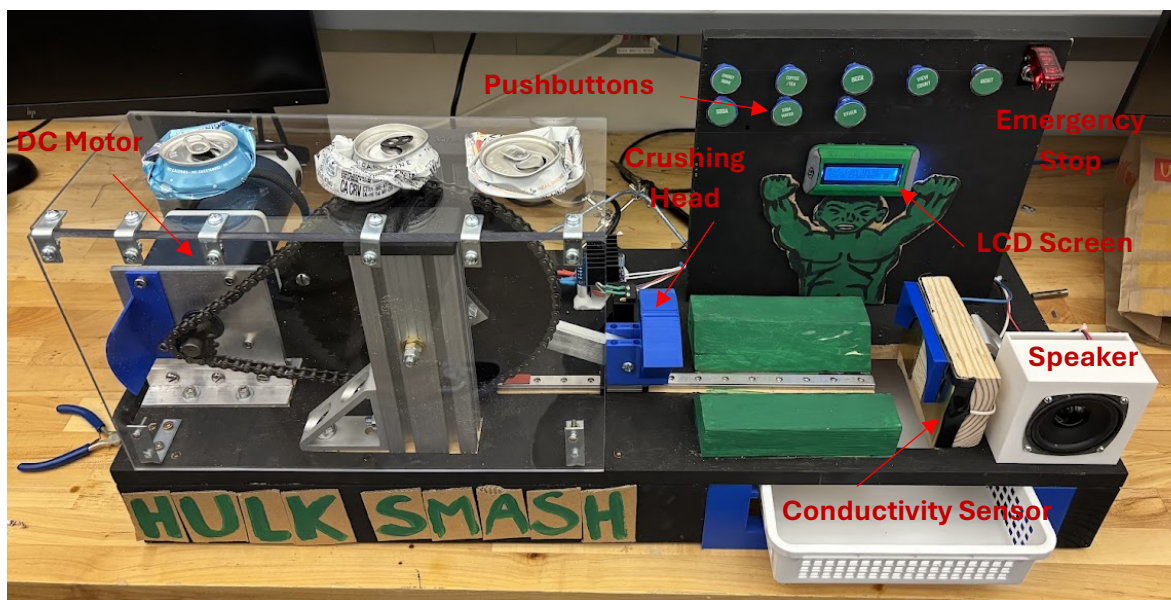


Figure SD3: Finished Assembly

HARDWARE

Hulk Smash's circuit diagram, shown in Figure SD4, illustrates the layout of the digital pins needed for all buttons, switches, sensors, LCDs, and modules. The number of digital pins required demonstrates the need for the Arduino Mega to manage all components effectively.

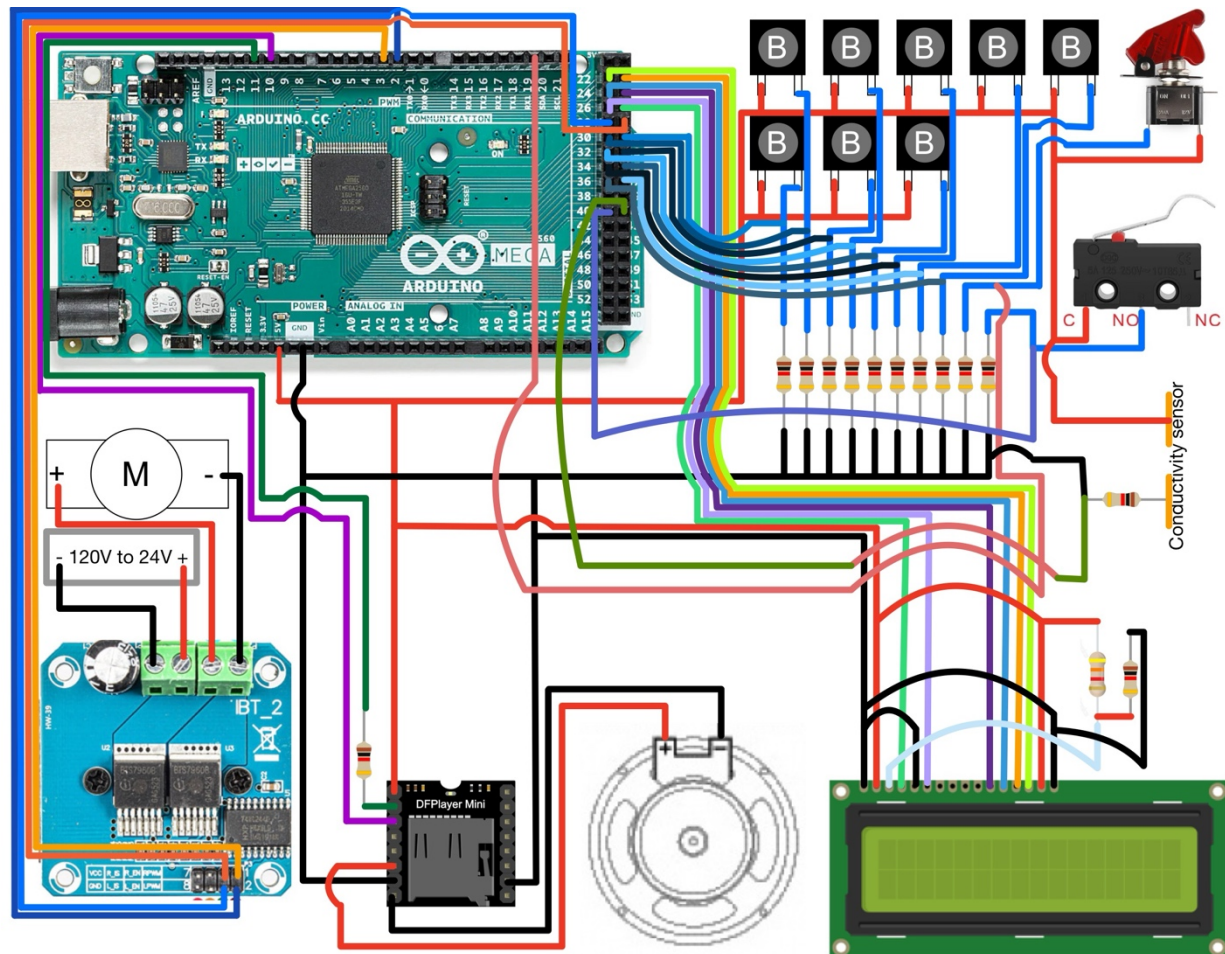


Figure SD4: Wiring Diagram

The flowchart in Figure SD5 outlines the code flow of the Hulk Smash system. Following the flowchart, a brief description of the system's operation is provided. The full code can be found in Appendix 1

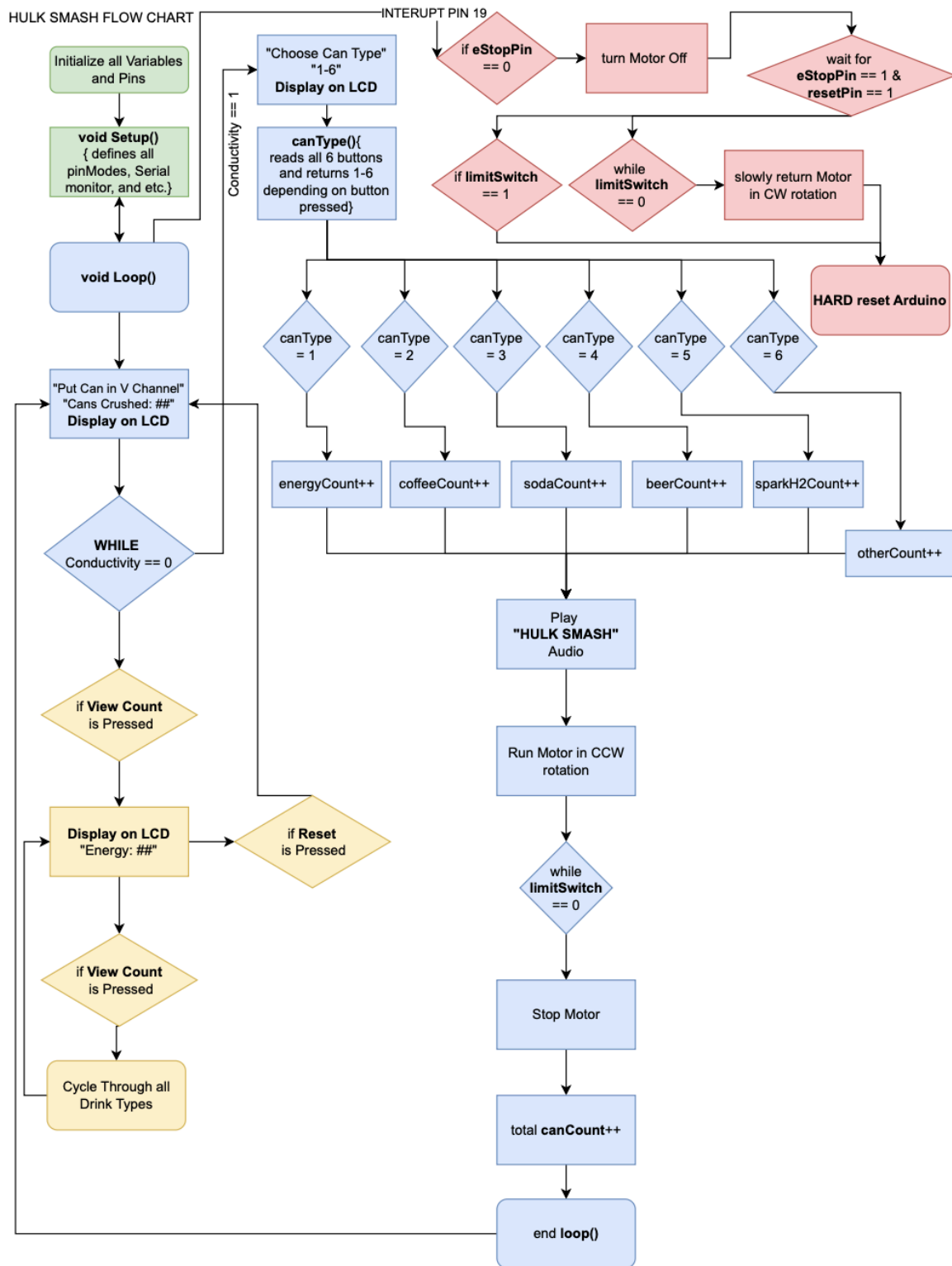


Figure SD5: Software Flowchart

Upon powering on the system, the LCD screen prompts the user with the message “Place Can in V Channel” on the top row and “Cans Crushed: ##” on the bottom row. The system then begins monitoring the conductivity sensor and the “View Count” button.

If the “View Count” button is pressed, the display cycles through the drink categories: “Energy: #”, “Coffee/Tea”, “Sparkling Water”, “Soda”, “Beer”, and “Other”. To crush a can, the user must first press the reset button to return to the home screen (“Place Can in V Channel”).

Once in the home screen mode, the user can place the can in the V channel, ensuring the bottom of the can makes contact with both sides of the conductivity sensor on the crushing backplate. After the can is correctly positioned, the user selects the drink type, after which they press one of the six available buttons to confirm the selection.

Upon selection, the Arduino triggers the “Hulk Smash” audio from the SD card using the DFPlayer Mini. A PWM signal is then sent to the IBT-2 Motor Controller, which drives the motor to rotate counterclockwise until the limit switch is activated. Once the limit switch is depressed, the motor stops.

In case of an emergency, the E-Stop rocker switch can be pressed. When the rocker switch is triggered via an interrupt pin on the Arduino, the motor halts immediately. To resume normal operation, the rocker switch must be rearmed, and the reset button must be pressed. The reset will send a PWM signal to the motor, rotating it clockwise at a slow speed to allow the user time to safely address any issues or remove the can from the system.

DESIGN EVALUATION

The Hulk Smash Can Crusher successfully integrates functioning elements across all six required categories, exceeding expectations for each. The components are fully interfaced with an Arduino Mega microcontroller, ensuring cohesive system operation rather than isolated functionality.

Output Display

The device features an LCD screen that displays the total number of cans crushed. This provides real-time user feedback and stores cumulative data over multiple uses. The display is clear, responsive, and updates automatically following each crushing cycle, satisfying the requirements for a functional and informative output interface. The LCD is dynamic throughout the crushing process, displaying a countdown leading up to can crushing and “HULK SMASH!” during crushing. It provides instructions for users during the process, including where to place the can and when to select can type buttons. When the E-Stop is triggered, it tells the user how to proceed and reset the machine to continue crushing. Finally, the LCD can also display the totals for each can type crushed throughout the machine’s history.

Audio Output

A speaker is incorporated into the design and delivers a digitally pre-recorded voice message during operation. The audio played was stored on a micro-SD card in an MP3 file format. This goes beyond a basic buzzer by enhancing the user experience with entertainment. The speaker is powered through a microcontroller and operates at a sufficient volume for clear

audibility. This implementation ranks in the upper tier of the category due to its software-controlled audio integration.

Manual User Input

The system uses a plethora of digitally open switches interfaced with the Arduino to initiate the crushing cycle. Throughout the process, there are multiple opportunities for manual user input. The input is required before any automatic operation begins, ensuring safety and intentional use. These buttons are also used to explore the stored information in the system, and as safety precautions. To excel in this category, Hulk Smash ensured there were multiple scenarios where manual user input was utilized.

Automatic Sensors

The device utilizes multiple automatic sensors throughout operation. To begin, a conductivity sensor (functionally like a limit switch) detects the presence of an aluminum can. When a can completes the circuit, it triggers the microcontroller to begin the crushing cycle. If a foreign object like glass or a water bottle were to be placed in the channel, the sensor would not allow for the operation to continue. This automatic sensing system ensures proper can positioning without requiring additional user input, meeting the category requirements with a simple yet effective sensing solution.

In addition, a limit switch was also used in the system to locate the crushing head effectively and ensure the mechanism always returned to the correct starting position after crushing. The limit switch was placed at the location where the crushing head is farthest from the crushing back plate. It is triggered when the back of the crushing head closes the switch. Due to

the mechanical design of the system, this would occur after a can has been crushed and the mechanism is traveling in the reverse direction. Once crushing has completed and the head triggers the limit switch, the can count is able to be updated. This sensor also allows for proper re-homing if the E-STOP was pressed, and the machine needed to return to its original position when the reset button is pressed.

Hulk Smash decided to excel in the automatic sensor category by automatic sensors in multiple processes and scenarios for this machine.

Actuators, Mechanism, and Hardware

The can crushing mechanism is powered by a 24V reversible DC motor connected to a crank-slider geartrain. The mechanical system features an 8-inch stroke to accommodate a range of can sizes and employs a 6:1 gear ratio for efficient torque output. The gear system and frame demonstrate thoughtful mechanical design with a mix of 3D-printed and machined components.

With the mechanism, there were many thoughtful additions added to ensure that cans could be crushed consistently and effectively. As mentioned previously, there was testing, and validation conducted to ensure the motor would be able to generate enough force to crush cans. The crushing head, can channel, and crushing back plate were also thoughtfully designed to ensure the can was not launched from the channel during crushing. These details represent a robust and reliable mechanical integration worthy of a strong score in this category.

Logic, Processing, Control, Miscellaneous

The system is controlled using an Arduino-based open-loop program. It handles input from the conductivity sensor, processes commands from the many manual user inputs, drives the motor, plays pre-recorded audio, and updates the LCD display. The system also includes basic data storage to maintain a running tally of crushed cans, as well as advanced data storage by keeping a running total for each type of can crushed via the machine. An Arduino interrupt function was also included in the code for the emergency stop. This ensured that no matter what was happening in the main loop, machine operation could always be terminated immediately for safety.

Another unique thing the note about this system is that the conductivity sensor was not purchased but made by Hulk Smash. This highlights engineering ingenuity and affordability. The team also made sure to maximize affordability by utilizing resources in the engineering building. All stock was acquired from the CSU EMEC, and many components were reused from the free Arduino kits provided in the classroom. To also maximize affordability, compression testing, and force analysis were conducted to make sure the most effective and affordable motor was selected. While crushing a can did require a very large and costly motor, it was made sure that there was data justification as to why each selected specification was necessary.

Grading Adjustments

Below, a table is constructed based off the group's opinion on grading adjustments due to overall device design and performance.

Category	Grading Adjustments	Justification
Proposal	+5	The proposal was completed in a professional and through manner.
Design Notebook	+10	The design notebook was always organized and on time. In addition, there were many tasks completed that were not directly listed but done to ensure the effectiveness of the machine.
Final Report	+10	Extreme thoroughness and detail were applied to this report to give a cohesive synopsis of the function of the machine and the work done throughout the semester.
Construction Quality, Aesthetics, Consumer Appeal	+10	The whole machine was thoughtfully constructed with aesthetic I mind. The mechanism was intentionally covered by clear acrylic to display the gear system. The whole project was decorated in theme with name, Hulk Smash. Even the back included decorations of crushed cans. A handle system was also added for easy transportation of the machine. The machine was also able to lay flat on any surface.
Level of Effort	+10	The team made sure to contribute at least 10 hours a week combined very week to ensure an

		exceptional and complete project was delivered on presentation day. The performance on evaluation day and the design notebook throughout the semester are the best testaments to the level of effort put into this project by this team.
Integration	0	Everything was well integrated and worked effectively.
Cost	+5	While the motor was expensive due to the high torque requirement, cost was minimized as much as possible. All stock was acquired for free, as well as electrical components.
Demonstration Performance	0	All can types were effectively and efficiently crushed and disposed of during presentation. Every described feature was also working properly during the demonstration.
Assembly/Safety	0	Safety covers were installed around moving parts to make sure that no user could potentially get trapped in the machine. There was also an emergency stop placed for immediate machine termination. All stock was properly sanded and smoothed to ensure no one could accidentally be cut by any component.

Controller Selection	0	The team used an Arduino Mega, due to the many different components and pins necessary, but no extra microcontrollers were unnecessarily used.
Power Sources	0	The main power source was a wall outlet, which is easily found in buildings. The Arduino was powered by a 9V battery, which was distributed in lab kits.
Early Bird	+5	The team successfully completed the early bird.
TOTAL: + 55 Points		

PARTIAL PART LIST

Gear Train System: 6:1 Gear Ratio

To achieve the desired crushing speed and force to successfully crush three formats of aluminum cans, a speed reducer gear system was implemented. A speed reducer gear train was used to increase the amount of output torque based on the input torque on the motor.

12T Input Gear

The 12-tooth hardened steel input gear fits directly onto the shaft of the DC motor. This gear was chosen because of its proper fitment onto the motor shaft, affordability and durability. An image of the 12-tooth input gear is shown below in Figure PP1.

Vendor:	Model Number:	Price:
<u>RHKING</u>	B0BV1JLM57	\$15.99



Figure PP1: 12T Input Gear

72T Output Gear

The 72-tooth output gear is mounted onto a CNC milled aluminum adapter plate (Figure PP3) which connects to a support block allowing the gear to rotate freely. A crank slider mechanism was used to crush the aluminum cans once placed in the machine. This mechanism was inspired by a piston-cylinder machine commonly found in an internal combustion engine. Selecting this larger 8.750-inch diameter gear allowed for a pin to be machined on the gear 4 inches from the center of the gear to crush cans that are 8 inches in length. Once the gear made a full revolution the stroke of the connecting rod would displace 8 inches which provides enough room to crush a 24 fluid oz. aluminum can. The 72T output gear is shown in Figure PP2.

Vendor:	Model Number:	Price:
<u>Gekufa</u>	B0B79C41WN	\$23.99



Figure PP2: 72T Output Gear

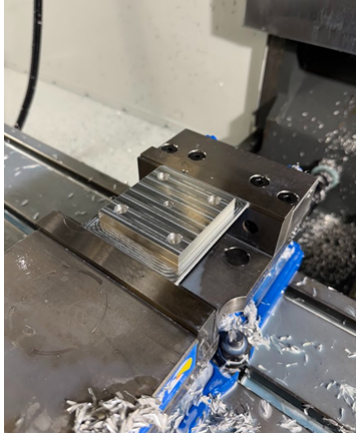


Figure PP3: Adapter Plate for 72T Output Gear

Chain: Number 35 Carbon Steel Roller Chain

A #35 roller chain was used to transfer rotational motion from the input gear to the output gear shown in Figure PP4. Using this industrial chain provided adequate rigidity and strength to crush a variety of can sizes ranging from 7.5 fluid oz. to 24 fluid oz.

Vendor:	Model Number:	Price:
<u>AZSSMUK</u>	662225027820	\$9.99



Figure PP4: Number 35 Roller Chain

Motor: Brushed 24V 150W 100RPM DC Motor

There were many options for selecting an actuator to crush an aluminum can. Some of the options considered were stepper motors, linear actuators, solenoids, and pneumatic actuators. Ultimately, ease of manufacturability, affordability, and component features narrowed the selection to a DC motor using a crank-slider mechanism.

A 24 Volt 150-Watt direct current motor was selected based on force data, gear ratio calculations, and one of the two design constraints for the crank-slider gear train. Being able to crush a can in under 5 seconds while delivering enough torque to drive a connecting rod and crushing head took a significant amount of design time. Since this was the most expensive component in the entire assembly, preliminary and critical design reviews were held to select the best motor for this application.

Crushing data was collected using a uniaxial force tester to determine the maximum amount of force required to crush three different formats of aluminum cans shown in Figure AP1. It was found that the maximum crushing force to crush an empty aluminum can from its original form to 1.25 inches is 149lbs (Figure AP2). There was a total of nine tests with all three formats of aluminum can to ensure proper data was being collected. This force data was used in a calculator shown in Figure AP3 which considered, max power output, max RPM, input and output torque, and time taken to crush the can. A load factor of safety (FOS) of 1.55 was used to account for unknown friction in the mechanical system along with efficiency losses inside the

DC motor. With a 6:1 gear ratio, the 100RPM motor driving the input gear provided enough rotational speed to the output gear to crush a can in 2.8 seconds on average.

Vendor:	Model Number:	Price:
<u>Stepper Online</u>	F5D150-24GU- 30S/5GU30KB	\$86.67



Figure PP5: 24V DC Motor

HARDWARE

Motor Controller: IBT-2 H Bridge

The IBT-2 H Bridge motor controller utilizes an H-Bridge circuit with transistors to efficiently control the direction and speed of a DC motor. By receiving Pulse Width Modulation (PWM) signals from the Arduino, the IBT-2 adjusts the motor's rotation, enabling precise control over its motion. The controller is capable of handling up to 24V DC, allowing it to power the motor with varying duty cycles determined by the PWM signal. This functionality ensures

smooth operation and fine-tuned motor speed adjustments, making it an essential component for controlling the motor's performance in the can crushing system. While there are various motor controllers available, this specific model was selected during the early stages of the project because a team member already owned one, and it proved to be a perfect fit for the system's needs.

Vendor:	Model Number:	Price:
<u>Amazon</u>	IBT-2	\$6.00

10T85 Lever Arm Limit Switch – SPDT

The 10T85 Limit Switch was implemented in the Hulk Smash system as a position sensor to detect the home position of the crank arm in the mechanical system. This sensor provided the system with a reliable reference point, enabling it to determine whether the mechanical system had completed a full revolution. The limit switch proved especially valuable after E-stop activation, as the mechanical system would shut off instantly, potentially leaving the system at any point in its revolution. By using the limit switch, the system could accurately reset to a known position. The limit switch was generously provided by the Nelson Lab Coordinator.

Vendor:	Model Number:	Price:
<u>DigiKey</u>	MS0850502F01S1A	\$1.14

Conductivity sensor

The conductivity sensor in the can crushing system consisted of two brass plates mounted on the backing of the crushing wall. These plates were strategically positioned so that they did

not touch unless a can was present, effectively completing the circuit. One plate was connected to a 5V supply, while the other was linked to a pull-down resistor and a digital pin on the Arduino, allowing it to read a discrete signal of either zero or one. When a can was inserted and the circuit closed, it sent a “1” signal to the Arduino. This signal prompted the LCD screen to ask the user to select the type of can being crushed, options included energy drinks, coffee/tea, soda, sparkling water, other, and beer. This step helped the system track the types of cans being processed without altering the motor’s voltage or PWM requirements, thus enhancing data collection and user interaction.

E-Stop Rocker Switch

The E-Stop rocker switch was strategically placed in the top right corner of the control panel and equipped with a protective cover plate. When the cover was closed and the switch was toggled to the open position, it immediately halted the motor, regardless of its position in the rotational cycle. This immediate stop ensured maximum safety by preventing any further movement until the system was rearmed. To resume operation, the switch had to be armed again, and a separate normally open reset button located to its left had to be pressed. Upon pressing the reset button, the motor would slowly reverse at a lower RPM, allowing users ample time and control to safely address any issues or remove themselves from the system. This thoughtful design significantly enhanced the safety and user-friendliness of the can crushing project.

Vendor:	Model Number:	Price:
<u>Amazon</u>	90173E	\$2.40

DFplayer – Mini MP3 Player

The DFPlayer Mini MP3 player module was an integral part of the can crushing system, adding a unique auditory element to the user experience. The module used an SD card to store MP3 files and was connected to the Arduino through a breadboard with jumper wires for easy integration. Controlled by the DF Robot library in the Arduino IDE, the DFPlayer Mini allowed for straightforward coding and rapid implementation of sound. When a user placed a can in the channel and selected the type of can to be crushed, the DFPlayer Mini played a “Hulk smash” audio clip, signaling that the motor was about to start. This fun audio cue enhanced the user experience and added an engaging, thematic element to the project. Once the audio clip finished, the motor activated, and the can crushing mechanism completed its cycle in just a few seconds.

Vendor:	Model Number:	Price:
<u>Amazon</u>	DFR0299	\$9.90

16x2 Liquid Crystal Display

The 16x2 LCD played a crucial role in guiding the user through the can crushing process and displaying important information. In idle mode, the top row of the display instructed the user to “Put can in V Channel,” while the bottom row showed the total number of cans crushed. Users could press a “View Count” button to switch the display to a detailed breakdown of how many cans of each type had been processed since the last reset. When a can was placed in the channel and the circuit completed, the screen prompted the user to “Choose Type of Can: 1-6,” corresponding to the different drink categories. After the user made a selection, the LCD displayed “HULK SMASH” as the audio cue played and the crushing process began. Once the motor finished its cycle, the display returned to the idle screen, ensuring the user always had clear and relevant information.

Vendor:	Model Number:	Price:
<u>DigiKey</u>	DFR0063	\$11.90 INCLUDED

Normally Open - Button

The can crushing system utilized eight normally open buttons arranged in a unique configuration: the first six buttons were placed in a 2x3 grid to the left, each corresponding to one of the drink types selectable by the user. Adjacent to these, on the top row, were the final two buttons: the “View Count” button and the “Reset” button. The “View Count” button allowed users to see a detailed tally of the number of cans crushed for each type since the last reset. The “Reset” button, located on the far right near the rocker switch, served dual functions: it returned the display to the home screen from the view count window, and after an emergency stop, it initiated the motor’s slow return to the home position. This intuitive arrangement ensured that users could easily control and monitor the system at every step.

Vendor:	Model Number:	Price:
<u>Amazon</u>	N/A	10 for \$6.99

LESSONS LEARNED

Throughout the development of the Hulk Smash Can Crusher, there were several challenges that required creative problem-solving and iteration. One significant issue involved the gear alignment in the crank-slider mechanism. Initially, the system experienced mechanical binding due to poor alignment, which impeded smooth motion. To resolve this, the gear mounts were adjusted to make sure the setup was completely rigid, using spacers and fasteners to ensure correct gear meshing. For future students, it is recommended that building and testing mechanical subsystems is done as early as possible, especially under load, to identify misalignments before final assembly. By testing the mechanical system early, the team was able to readjust and create a reliable mechanism.

Another challenge arose with the conductivity sensor used for can detection. In early testing, the sensor was unreliable due to inconsistent contact and sensitivity variations. The sensor was redesigned with large thin brass plates to increase surface area for contact and provide a flat resting place for the can. Future teams should rigorously test sensor performance under real operating conditions and use indicator LEDs or serial printouts during debugging to verify consistent behavior.

Lastly, some issues were faced when first trying to run the system with all the components. Each component was initially tested individually, but when combined, sometimes the code would freeze at certain steps. Loose jumper wires would occasionally disconnect as well. This was solved by code debugging, soldering critical connections and reinforcing them with cable ties. Future builders should anticipate challenges integrating individual components and leave time to solve problems that may occur when trying to assemble the whole system.

Overall, the largest advice would be to plan to complete the project with at least three weeks left to spare. Often, the timeline will get readjusted based on roadblocks encountered that were not previously planned for. However, when those issues occur, try to keep working on other tasks and talk to others to help trouble shoot. Many classmates will also be going through the same problems, and being collaborative helps solve issues efficiently.

APPENDIX

[1]



Figure AP1: Force Data Collection

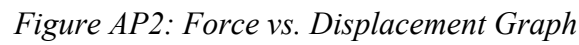


Figure AP3: Gear and Load Calculator

30

MECH307_FINAL.ino

```
1  /*Compiled Code for Mechatronics Project
2  Updated: 5-4-2025
3  Authors: Brenton Yasuoka */
4
5  /*
6  Updates: 5/4/25
7  Adding 6 Drink Buttons, 3 count buttons (view, increase, decrease),
8  Count View Screen for each type of drink
9  -> Increase and Decrease changes the overall count and the inividual can count depending on what
10     | type of can is on the screen
11
12  When the conductivity sensor is in place it should now prompt the user to select one of the 6 types of cans
13
14  Adding the audio DFplayer code as well
15  Depending on the canType selected a specific audio from the can will be played
16
17  Commented out JokeDisplay Function because the Joke will be the Audio
18  */
19
20  //Define variables and Libraries
21  int canCount = 0; // Counter for crushed cans
22  int energyCanCount = 0;
23  int sparkWaterCanCount = 0;
24  int otherCanCount = 0;
25  int coffeeTeaCanCount = 0;
26  int sodaCanCount = 0;
27  int beerCanCount = 0;
28
29  //Buzzer variables and Libraries
30  #include <pitches.h>
31
32  //DFPlayer variables and Libraries
33  #include "SoftwareSerial.h"
34  #include "DFRobotDFPlayerMini.h"
35
36  int dfCheckLED = 33;
37  SoftwareSerial mySoftwareSerial(10, 11); // RX, TX
38  DFRobotDFPlayerMini myDFPlayer;
39  int setList[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

MECH307_FINAL.ino

```
40
41 //LCD Screen variables
42 #include <LiquidCrystal.h> //LCD Screen library
43 LiquidCrystal lcd(27, 26, 25, 24, 23, 22); // Initialize LCD with interface pins (RS, E, D4, D5, D6, D7)
44
45 //Can Type Selection Buttons
46 int canType = 0;
47 bool buttonState1 = 0;
48 bool buttonState2 = 0;
49 bool buttonState3 = 0;
50 bool buttonState4 = 0;
51 bool buttonState5 = 0;
52 bool buttonState6 = 0;
53
54 #define button1Pin 39 // Define the pins for each button
55 #define button2Pin 36
56 #define button3Pin 34 // Need to find a pin we can use
57 #define button4Pin 37
58 #define button5Pin 38
59 #define button6Pin 35
60
61 //Conductivity Sensor Pin
62 #define conductivitySensorPin 40
63
64 //Can Type Count Buttons
65 bool buttonState7 = 0;
66 bool buttonState8 = 0;
67 bool buttonState9 = 0;
68 int goToCanView = 0;
69
70 #define button7Pin 32 //ViewCanType button
71 #define button8Pin 33 //Increase
72 #define button9Pin 30 //Decrease
73
74 //Extra Button
75 bool buttonState10 = 0;
76 #define resetPin 31
77
78 // E Stop Button Definitions
79 #define EstopPin 19 //Blue Button
```

MECH307_FINAL.ino

```

80  volatile bool button1Pressed = false; // Flag for button 1 press
81  volatile bool button2Pressed = false; // Flag for button 2 press
82
83  //Motor Pin Definitions
84  #define R_EN 28 // Right Enable Pin
85  #define L_EN 29 // Left Enable Pin
86  #define R_PWM 2 // Right PWM Pin
87  #define L_PWM 3 // Left PWM Pin
88
89  //Limit Switch Pins
90  #define limitSwitchPin 41
91  bool homeState = 0;
92  int initialHome = 0;
93
94  void motorCrushing(){
95      digitalWrite(R_EN, HIGH); // Enable Right Motor
96      digitalWrite(L_EN, HIGH); // Enable Left Motor
97      Serial.println("Motor Crushing started");
98      analogWrite(R_PWM, 0); // Run right motor at ~60% speed
99      analogWrite(L_PWM, 200); // Run left motor at ~80% speed
100     delay(1000);
101     homeState = digitalRead(limitSwitchPin);
102     while(homeState == !homeState){
103         homeState = digitalRead(limitSwitchPin);
104     } //Time delay will need to be determined on how many RPMs we
105     //are getting out so it only makes x amount of rotations that we want
106     while(homeState == 1){
107         homeState = digitalRead(limitSwitchPin);
108     }
109     analogWrite(R_PWM, 0);
110     analogWrite(L_PWM, 0);
111     digitalWrite(R_EN, LOW); // Enable Right Motor
112     digitalWrite(L_EN, LOW); // Enable Left Motor
113 }
114
115 void E_STOP() {
116     button1Pressed = true; // Set flag when Button 1 is pressed
117     Serial.println("E_STOP Pressed");
118     analogWrite(L_PWM, 0);
119     analogWrite(R_PWM, 0);

```

MECH307_FINAL.ino

```
120     button2Pressed = digitalRead(resetPin);
121     button1Pressed = digitalRead(EstopPin);
122     while(button1Pressed == 0){
123         button1Pressed = digitalRead(EstopPin);
124         delay(25);
125     }
126     while(button2Pressed == 0){
127         button2Pressed = digitalRead(resetPin);
128         Serial.println(button2Pressed);
129         delay(25);
130     }
131     motorHomeCode();
132 }
133
134 void motorHomeCode(){
135     while(initialHome == 0){
136         homeState = digitalRead(limitSwitchPin); //Check Limit switch if its 1 then we are home if its 0 then we are not
137         if (homeState == 1){ //if its 1 can start crushing again (maybe eventually ask did the can crush if it did then
138             //don't do anything if it didn't decrease can count and ask the user to restart)
139             //placeCanDisplay();
140             homeState = 0;
141             initialHome = 1;
142             lcd.clear();
143             lcd.setCursor(0, 0); // Adjust based on a 16-character display
144             lcd.print("Must Hit Arduino"); //11 characters
145             lcd.setCursor(1, 1); // Adjust based on a 16-character display
146             lcd.print("Reset Button");
147             delay(5000);
148             //loop();
149             //return;
150         }
151         else{ //if its 0 turn motor right to bring back to start.
152             Serial.println("Motor Returning");
153             while(homeState != 1){
154                 digitalWrite(R_EN, HIGH); // Enable Right Motor
155                 digitalWrite(L_EN, HIGH); // Enable Left Motor
156                 analogWrite(R_PWM, 50); // Run right motor at ~60% speed
157                 analogWrite(L_PWM, 0); // Run left motor at ~80% speed
158                 delay(15); //just for debouncing i think but don't want it too slow so might reduce this
159                 homeState = digitalRead(limitSwitchPin);
160             }
161         }
162     }
163 }
```

MECH307_FINAL.ino

```
161
162     analogWrite(R_PWM, 0); // Run right motor at ~60% speed
163     analogWrite(L_PWM, 0); // Run left motor at ~80% speed
164     //delay(15);
165 }
166 }
167 }
168
169 void canTypeIncrease(int canType){
170     if (canType == 1){
171         energyCanCount++;
172     }
173     else if(canType == 2){
174         sparkWaterCanCount++;
175     }
176     else if(canType == 3){
177         coffeeTeaCanCount++;
178     }
179     else if(canType == 4){
180         beerCanCount++;
181     }
182     else if(canType == 5){
183         sodaCanCount++;
184     }
185     else if(canType == 6){
186         otherCanCount++;
187     }
188 }
189
190 void viewCanType(int initial){
191     lcd.clear();
192     while(digitalRead(resetPin) == 0){
193         lcd.clear();
194         if (initial == 1){
195             lcd.setCursor(0,0);
196             lcd.print("Energy:"); //7 Characters
197             lcd.setCursor(0,1);
198             lcd.print(energyCanCount);
199             if (digitalRead(button8Pin) == 1){
200                 energyCanCount++;
201             }
```

MECH307_FINAL.ino

```
202         else if (digitalRead(button9Pin) == 1){
203             energyCanCount--;
204         }
205         if (digitalRead(button7Pin) == 1){
206             initial++;
207         }
208     }
209     else if (initial == 2){
210         lcd.setCursor(0,0);
211         lcd.print("Sparkling Water:"); //7 Characters
212         lcd.setCursor(0,1);
213         lcd.print(sparkWaterCanCount);
214         if (digitalRead(button8Pin) == 1){
215             energyCanCount++;
216         }
217         else if (digitalRead(button9Pin) == 1){
218             energyCanCount--;
219         }
220         if (digitalRead(button7Pin) == 1){
221             initial++;
222         }
223     }
224     else if (initial == 3){
225         lcd.setCursor(0,0);
226         lcd.print("Coffee/Tea:"); //7 Characters
227         lcd.setCursor(0,1);
228         lcd.print(coffeeTeaCanCount);
229         if (digitalRead(button8Pin) == 1){
230             energyCanCount++;
231         }
232         else if (digitalRead(button9Pin) == 1){
233             energyCanCount--;
234         }
235         if (digitalRead(button7Pin) == 1){
236             initial++;
237         }
238     }
239     else if (initial == 4){
240         lcd.setCursor(0,0);
241         lcd.print("Beer:"); //7 Characters
```


MECH307_FINAL.ino

```
241     lcd.print("Beer: "); // 7 Characters
242     lcd.setCursor(0,1);
243     lcd.print(beerCanCount);
244     if (digitalRead(button8Pin) == 1){
245         energyCanCount++;
246     }
247     else if (digitalRead(button9Pin) == 1){
248         energyCanCount--;
249     }
250     if (digitalRead(button7Pin) == 1){
251         initial++;
252     }
253 }
254 else if (initial == 5){
255     lcd.setCursor(0,0);
256     lcd.print("Soda:"); //7 Characters
257     lcd.setCursor(0,1);
258     lcd.print(sodaCanCount);
259     if (digitalRead(button8Pin) == 1){
260         energyCanCount++;
261     }
262     else if (digitalRead(button9Pin) == 1){
263         energyCanCount--;
264     }
265     if (digitalRead(button7Pin) == 1){
266         initial++;
267     }
268 }
269 else if (initial == 6){
270     lcd.setCursor(0,0);
271     lcd.print("Other:"); //7 Characters
272     lcd.setCursor(0,1);
273     lcd.print(otherCanCount);
274     if (digitalRead(button8Pin) == 1){
275         energyCanCount++;
276     }
277     else if (digitalRead(button9Pin) == 1){
278         energyCanCount--;
279     }
280     if (digitalRead(button7Pin) == 1){
```

MECH307_FINAL.ino

```
281         initial++;
282     }
283 }
284 else {}
285 if (initial == 7){
286     initial = 1;
287 }
288     Serial.println(initial);
289     delay(1000);
290 }
291 placeCanDisplay();
292 return;
293 }
294
295 void setup() {
296     // put your setup code here, to run once:
297     // Serial communication with the module
298     mySoftwareSerial.begin(9600);
299     // Initialize Arduino serial
300     Serial.begin(9600);
301     Serial.println("Setting Up...");
302     //Serial.begin(9600);
303
304     //Buzzer Setup Commands
305     //pinMode(buzzer, OUTPUT);
306
307     //LCD Setup Commands
308     lcd.begin(16, 2); // Set up the LCD's number of columns and rows
309     placeCanDisplay(); // Initial display
310
311     //Can Type Selection Setup Commands
312     pinMode(button1Pin, INPUT);
313     pinMode(button2Pin, INPUT);
314     pinMode(button3Pin, INPUT);
315     pinMode(button4Pin, INPUT);
316     pinMode(button5Pin, INPUT);
317     pinMode(button6Pin, INPUT);
```

MECH307_FINAL.ino

```

319 //Can Type Count Setup Commands
320 pinMode(button7Pin, INPUT);
321 pinMode(button8Pin, INPUT);
322 pinMode(button9Pin, INPUT);
323
324 //Motor Setup Commands
325 pinMode(R_EN, OUTPUT);
326 pinMode(L_EN, OUTPUT);
327 pinMode(R_PWM, OUTPUT);
328 pinMode(L_PWM, OUTPUT);
329
330 digitalWrite(R_EN, LOW); // Enable Right Motor
331 digitalWrite(L_EN, LOW); // Enable Left Motor
332
333 //E STOP Setup Commands
334 attachInterrupt(digitalPinToInterrupt(EstopPin), E_STOP, FALLING); // Button 1 (ramp down)
335
336 //DFPlayer Mini setup
337 while (!myDFPlayer.begin(mySoftwareSerial)) {
338
339     Serial.println(F("Not initialized:"));
340     Serial.println(F("1. Check the DFPlayer Mini connections"));
341     Serial.println(F("2. Insert an SD card"));
342
343     digitalWrite(dfCheckLED,1);
344 }
345
346 //digitalWrite(dfCheckLED,0);
347 myDFPlayer.setTimeout(500); // Serial timeout 500ms
348 myDFPlayer.volume(29);      // Volume 5
349 myDFPlayer.EQ(0);
350
351 Serial.println("Ready to Run");
352 }
353
354 void loop() {
355     // put your main code here, to run repeatedly:
356     //Screen is outputting a message to tell user to place can in the v channel
357     placeCanDisplay();
358     //If the View Can Count was pressed go to the new screen that allows for the user to see different can type counts

```

```

359
360 //wait for can to touch conductivity sensor - Once can touches sensor continue
361 conductivitySensor(); //this has a while loop in it that should not let us continue until we finish this function
362 selectCanDisplay(); //LCD Prompts to select can and audio prompts to select a can - wait for choice
363 canType = getButtonValue(0);
364 //int canType = 1;
365 //jokeDisplay(canType-1); //LCD shows comment on screen and audio joke plays the -1 is needed so that the first can type plays Joke 1
366 //jokeAudio(canType); //Plays the audio that was selected from the getButtonValue and the -1 is so that audio in positon 0 will start
367 myDFPlayer.play(1);
368 lcd.clear();
369 lcd.setCursor(3, 0); // Adjust based on a 16-character display
370 lcd.print("HULK SMASH!"); //11 characters
371 delay(4000);
372 motorCrushing();
373 //countDownDisplayAudio(); //LCD counts down and audio beeps like mario kart
374 canCount++;
375 canTypeIncrease(canType);
376 if (canCount == 100){ //Resets the canCount variable if it reaches 100 because we do not have enough space on the screen for 3 characters
377     canCount = 0;
378 }
379
380 }
381
382
383 //Place a can and # of cans crushed : display function
384 void placeCanDisplay() {
385     lcd.clear();
386
387     // Print "CRUSHING!" centered
388     lcd.setCursor(0, 0); // Adjust based on a 16-character display
389     lcd.print("Put Can in V"); //12 characters
390
391     // Print "Cans Crushed: ###"
392     lcd.setCursor(0, 1);
393     lcd.print("Cans Crushed: "); //13 Characters
394     lcd.setCursor(14, 1); // Move cursor to the right for count display
395     lcd.print(canCount); //1-2 Characters
396 }
397

```

MECH307_FINAL.ino

```
398 //Asks user to select a can : display function
399 void selectCanDisplay() {
400     //Serial.println("Selecting Can");
401     lcd.clear();
402
403     // Print "CRUSHING!" centered
404     lcd.setCursor(0, 0); // Adjust based on a 16-character display
405     lcd.print("Select Type Can"); //15 Characters
406
407     // Print "Cans Crushed: ####"
408     lcd.setCursor(2, 1);
409     lcd.print("Choose 1-6"); //10 Characters
410 }
411
412 //Count down and hulk smash : display function
413 void countdownDisplayAudio() {
414     lcd.clear();
415     Serial.println("Count Down Display Audio Initiated");
416     lcd.setCursor(2,0);
417     lcd.print("HULK SMASH!");
418     motorCrushing();
419     Serial.println("Count Down Display DONE!");
420     return;
421 }
422
423
424 //Can selection via Button : returns the selection for joke and display update
425 int getButtonValue(int initial) {
426     Serial.println("Selecting Can");
427     while (initial == 0){
428         buttonState1 = digitalRead(button1Pin);
429         buttonState2 = digitalRead(button2Pin);
430         buttonState3 = digitalRead(button3Pin);
431         buttonState4 = digitalRead(button4Pin);
432         buttonState5 = digitalRead(button5Pin);
433         buttonState6 = digitalRead(button6Pin);
```

MECH307_FINAL.ino

```
434
435     if (buttonState1 == HIGH) {
436         initial = 1;
437     }
438     else if (buttonState2 == HIGH) {
439         initial = 2;
440     }
441     else if (buttonState3 == HIGH) {
442         initial = 3;
443     }
444     else if (buttonState4 == HIGH) {
445         initial = 4;
446     }
447     else if (buttonState5 == HIGH){
448         initial = 5;
449     }
450     else if (buttonState6 == HIGH){
451         initial = 6;
452     }
453     else {
454         initial = 0; // No button is pressed.
455     }
456     delay(50);
457 }
458 Serial.print("Initial: ");
459 Serial.println(initial);
460 return initial;
461 }
462
463 //Conductivity Sensor code waiting for value change
464 void conductivitySensor(){
465     goToCanView = 0;
466     bool buttonState = digitalRead(conductivitySensorPin); // Read pin state (0 or 1)
467     while (buttonState == 0){
468         buttonState = digitalRead(conductivitySensorPin); // Read pin state (0 or 1)
469         Serial.println(buttonState);
470         delay(100); // Small delay to avoid flooding the Serial Monitor
471         if (digitalRead(button7Pin) == 1){
472             viewCanType(1);
473         }
474     }
```